

Smart Cabin & Vehicle Health Monitoring System — Architecture Explanation v1.0

1. Introduction: System Architecture and Its Crucial Role in the ECU

Architecture dictates how a software system is organised and interconnected. It answers three fundamental questions: What are the individual components? What is the specific responsibility of each component? How do these components communicate efficiently and safely?

In this project, the architecture was meticulously designed to mirror a real-world automotive Electronic Control Unit (ECU) running **Adaptive AUTOSAR** software. By organizing the code into distinct horizontal layers, the system guarantees that each layer only communicates with the layer directly beneath it. This decoupling ensures that the system is highly extensible (e.g., adding a new sensor won't break the dashboard) and fully testable in isolation.

*[!NOTE] Think of it like a highly efficient factory pipeline. The **sensor floor** fabricates raw materials (sensor readings). The **quality assurance floor** inspects these materials (AlertManager, DtcManager). The **records office** securely files compliance reports (Logger, EdrRecorder). Finally, the **front office** displays the executive summary on a status board (Dashboard). Each floor executes a single responsibility and securely hands work off to the next.*

2. Component Organization: The 13-Module File Structure

The entire project operates across 13 distinct header and source file pairs. This structure has been significantly expanded from a basic MVP to support 10 advanced automotive subsystems seamlessly.

```

Project/
├─ include/
│   ├─ Sensor.hpp           ← Module 1: Core sensor definitions & randomized fault injection
│   ├─ Alert.hpp           ← Module 2: Strategy pattern-based alert evaluation engine
│   ├─ DtcManager.hpp      ← Module 3: OBD-II Diagnostic Trouble Codes (DTC) & freeze frames
│   ├─ Logger.hpp          ← Module 4: Generic templates for asynchronous file logging
│   ├─ EdrRecorder.hpp     ← Module 5: Crash-safe Event Data Recorder (flight recorder)
│   ├─ JsonParser.hpp      ← Module 6: Custom lightweight JSON parser (no external deps)
│   ├─ Dashboard.hpp       ← Module 7: Live ANSI terminal display & historical sparklines
│   ├─ ThreadManager.hpp   ← Module 8: RAII thread pools, mutexes, & safe message queues
│   ├─ EcuMonitor.hpp      ← Module 9: System watchdog monitoring ECM, BCM, BMS, & TPMS
│   ├─ OtaSimulator.hpp    ← Module 10: Over-The-Air firmware flashing & safety interlocks
│   ├─ CanBus.hpp          ← Module 11: 11-bit CAN frame packing & transmission simulation
│   ├─ DriverProfile.hpp   ← Module 12: Hot-swappable user preferences & dynamic thresholds
│   └─ ServiceBus.hpp      ← Module 13: SOME/IP style publish/subscribe service mesh
├─ src/
│   ├─ [Matching 12 .cpp files containing the concrete implementations]
│   ├─ main.cpp            ← Composition root: Orchestrates initialization and thread spawning
│   └─ tests.cpp           ← 10 automated test cases for boundary & integration validation
├─ logs/                  ← Output directory for vehicle_log.txt, crash_edr.bin, test_report.csv
├─ data/                  ← Input directory for config.txt, test_cases.json
└─ CMakeLists.txt         ← Cross-platform C++17 build instructions

```

3. Dependency Management: The Compile-Time Include Chain and Avoiding Circular Loops

When the C++ compiler builds the project, it must resolve class dependencies in a strict order. To prevent compile-time overhead and fatal infinite loops, this project enforces a rigid, one-way include chain.

3.1 The Hierarchical Chain

```

Sensor.hpp      - Includes NOTHING from this project (Foundation module).
    ↑ Included by
Alert.hpp       - Includes Sensor.hpp (Requires full sensor details to evaluate rule strategies).
    ↑ Included by
Logger.hpp      - Logs alerts, but only requires forward declarations where possible.
    ↑ Included by
ThreadManager.hpp - The conductor module. Forward-declares most subsystems to manage threads.
    ↑ Included by
main.cpp        - Includes ALL headers. This is the top-level composition root.

```

3.2 The Forward Declaration Rule

If `Alert.hpp` included `Logger.hpp`, and `Logger.hpp` included `Alert.hpp`, the compiler would enter an infinite loop. The architectural solution is the **forward declaration**.

```
// Inside ThreadManager.hpp:
class Sensor;           // Tells the compiler "This class exists elsewhere"
class AlertManager;
class VehicleStatistics;
class Dashboard;
```

Because `ThreadManager` only stores references or smart pointers to these objects, the compiler only needs to know the type *exists*. The heavy `#include "Sensor.hpp"` is deferred entirely to `ThreadManager.cpp` where the methods are actually invoked. This is a critical C++ architecture practice for massive codebases.

4. Hierarchical Design: The Four-Layer Adaptive AUTOSAR Architecture

The system is compartmentalised into distinct layers. Code in a specific layer is strictly prohibited from invoking code in a layer above it.

- LAYER 4: Infrastructure & Advanced Services** (`ThreadManager`, `Dashboard`, `OtaSimulator`, `CanBus`, `ServiceBus`, `EcuMonitor`) The top-most orchestration layer. Manages all asynchronous worker threads, drives the user interface, flashes firmware, routes service bus messages, and monitors hardware health.
- LAYER 3: Persistence, Data & Diagnostics** (`Logger`, `EdrRecorder`, `JsonParser`, `DtcManager`) The data backbone. Offloads heavy disk I/O to background threads via thread-safe queues. Manages crash telemetry buffering, OBD-II code registration, and parses configuration files.
- LAYER 2: Alert Rules & Safety Strategies** (`Alert`) The logic engine. Actively compares real-time telemetry against loaded driver profiles. Utilizes a dynamic `priorityScore` to elevate warnings (e.g., escalating an open door warning if the vehicle is in motion).
- LAYER 1: Core Foundation** (`Sensor`, `DriverProfile`) The raw data generators. Uses standard C++ `<random>` (Mersenne Twister) to simulate realistic vehicle states and environmental conditions. Requires absolutely zero knowledge of the layers above it.

5. Module Deep-Dive: Responsibilities and Key Components

5.1 Foundation (Sensors & Profiles)

Component	Responsibility	Implementation Details
<code>Sensor (Base)</code>	Interface contract	Defines polymorphic <code>update()</code> and <code>getNumericValue()</code>
<code>EngineTemperatureSensor</code>	Powertrain simulation	Ranges from 50–130°C with 5% fault injection probability
<code>DriverProfile</code>	Dynamic thresholding	Enables hot-swapping between Eco, Sport, and Default modes

5.2 Diagnostics & Safety

Component	Responsibility	Implementation Details
-----------	----------------	------------------------

Component	Responsibility	Implementation Details
AlertRule	Strategy engine	Decouples threshold logic from the main <code>AlertManager</code>
AlertManager	Centralized alert router	Tracks active alerts and maintains historical logs
DtcManager	Fault registry	Stores OBD-II P-codes (e.g., P0117) and captures Freeze Frames

5.3 Persistence & Asynchronous I/O

Component	Responsibility	Implementation Details
SafeQueue<T>	Thread-safe buffering	Uses <code>std::mutex</code> and <code>std::condition_variable</code> for non-blocking pushes
EventLogger<T>	Disk writing	Drains the <code>SafeQueue</code> on a background thread to prevent sensor lag
EdrRecorder	Crash preservation	Circular ring-buffer that automatically flushes on critical impact

5.4 Infrastructure & Comms

Component	Responsibility	Implementation Details
ThreadManager	Lifecycle management	Guarantees all threads are joined gracefully via RAII destructors
ServiceBus	Decoupled messaging	Allows modules to publish/subscribe to topics (e.g., "ota.status")
CanBus	Network simulation	Serializes float data into exact 11-bit ID and 8-byte CAN payloads
SharedVehicleData	Concurrency hub	Holds the <code>std::shared_mutex</code> that coordinates all cross-thread data access

6. Memory Management: Runtime Object Ownership and RAII Principles

Object lifecycles are explicitly managed to prevent memory leaks and ensure thread safety. No raw pointers or manual `new/delete` calls are used anywhere in the architecture.

Exclusive Ownership (`unique_ptr`)

Sensors are instantiated in `main.cpp` as a `std::vector<std::unique_ptr<Sensor>>` and then explicitly `std::move`'d into the `SharedVehicleData` hub. This guarantees zero copy overhead and ensures the vectors are destroyed automatically when the program terminates.

Shared Access (`shared_ptr`)

The `SharedVehicleData` struct itself is passed as a `std::shared_ptr` into every spawned worker thread. This guarantees that the core state (Sensors, AlertManager, Logger) remains alive in memory as long as at least one background thread is still executing.

Reader-Writer Synchronization (`shared_mutex`)

Instead of a standard `std::mutex` which locks out everyone, the architecture utilizes `std::shared_mutex`.

- **Sensor & Monitor Threads** take a `std::unique_lock` (exclusive write access) to update telemetry.
- **Dashboard Thread** takes a `std::shared_lock` (read-only access). This allows the dashboard to read data simultaneously with other readers without ever blocking the critical high-speed sensor writes.

7. Operational Pipeline: The Complete Real-Time Data Flow

The system runs asynchronously. Here is the exact lifecycle of a data point from generation to visualization:

1. **Generation (Sensor Thread — 500ms):** The background sensor thread wakes up, takes an exclusive write lock, and invokes `update()` on all polymorphic sensors. A random number generator simulates a sudden spike in engine temperature.
2. **Evaluation (Monitor Thread — 1000ms):** The monitor thread wakes up, takes a write lock, and processes the new readings through the `AlertRule` strategies. It detects the temperature spike violates the `DriverProfile` threshold.
3. **Escalation (Alert & DTC Managers):** An `Alert` object is instantiated with a high `priorityScore`. The `DtcManager` is immediately notified to lock in an OBD-II Freeze Frame of the exact moment of failure.
4. **Logging (Logger Thread — 1000ms):** The alert string is pushed into the `SafeQueue`. The `Logger` thread, operating completely off the main data lock, pops the string and executes the heavy disk I/O write to `vehicle_log.txt`.
5. **Visualization (Dashboard Thread — 2000ms):** The UI thread takes a read lock, safely accesses the active alerts list, and renders a flashing red warning and updated historical sparkline graph to the ANSI terminal.

8. Inter-Process Communication: How The Modules Talk To Each Other via SOA

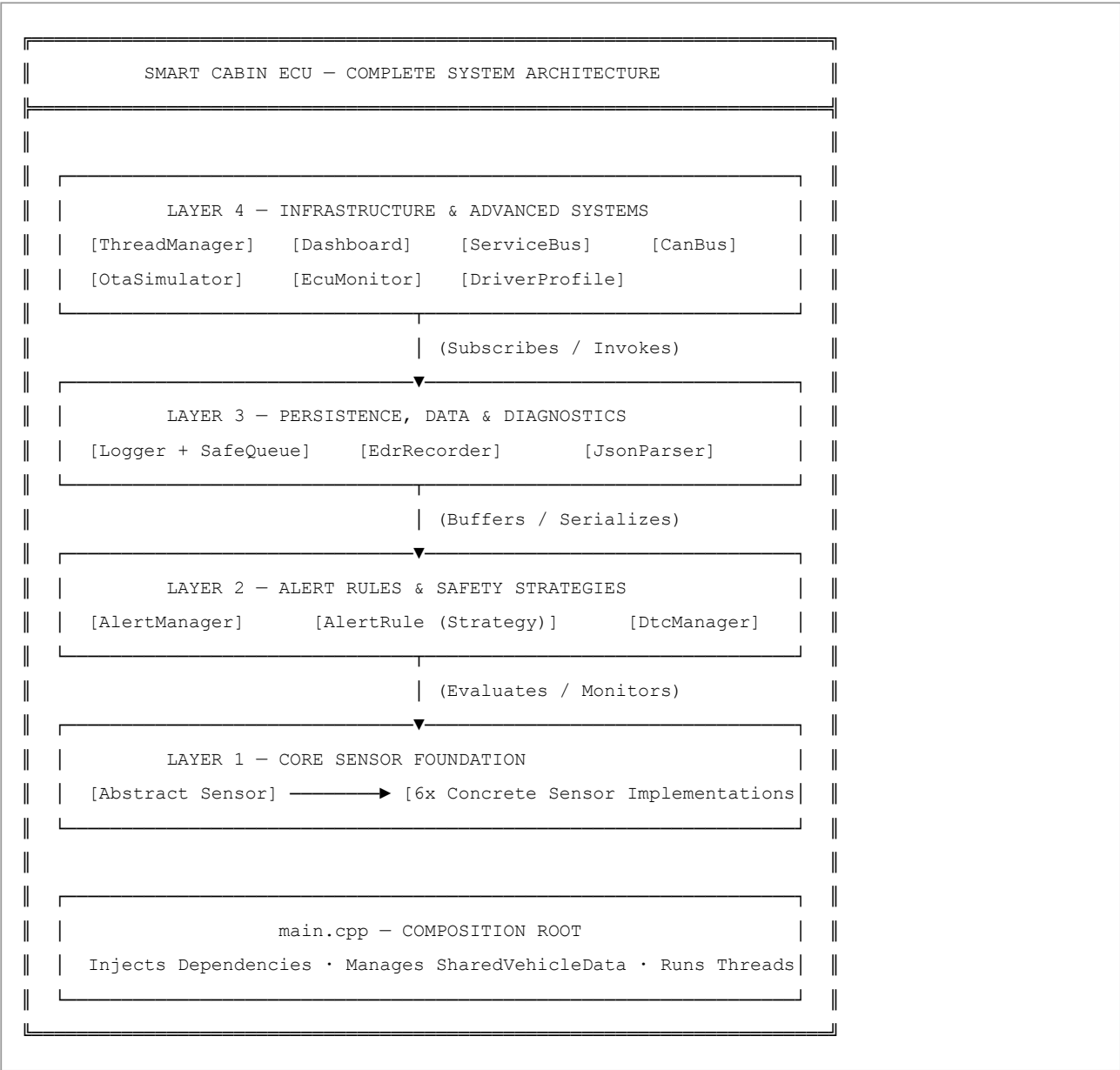
To prevent modules from becoming tightly coupled, the architecture implements a **Service-Oriented Architecture (SOA)** via the `ServiceBus`.

Instead of the `OtaSimulator` directly calling a method on the `Sensor` array to check the vehicle's speed before flashing firmware, it subscribes to a topic:

```
// OtaSimulator setup:
serviceBus->subscribe("vehicle.speed", [](double currentSpeed) {
    if (currentSpeed > 0) abortFirmwareUpdate();
});
```

The Monitor thread blindly publishes to this topic every second. This decoupled Publish/Subscribe mechanism mirrors exactly how modern Adaptive AUTOSAR systems use `ara::com` to communicate across different physical ECUs without hardcoded dependencies.

9. Visual Blueprint: Architecture Block Diagram



10. Industry Alignment: Real-World Automotive Mapping to AUTOSAR Standards

Every architectural decision in this simulation maps directly to production automotive software standards.

This C++ Component	Real-World Automotive Equivalent	AUTOSAR Standard / Spec
Sensor Framework	MCAL (Microcontroller Abstraction Layer)	Classic AUTOSAR MCAL
AlertManager	Vehicle Health Manager (VHM)	Adaptive AUTOSAR
Logger / EdrRecorder	Cockpit Domain Controller Blackbox	ISO 16054 (EDR requirements)
DtcManager	Diagnostic Event Manager (DEM)	AUTOSAR DEM SWC & OBD-II
ThreadManager	Execution Management State Machine	Adaptive AUTOSAR <code>ara::exec</code>
CanBus	CAN Communication Stack & PDU Router	AUTOSAR RTE / COM
ServiceBus	SOME/IP Service-Oriented Communications	Adaptive AUTOSAR <code>ara::com</code>

This C++ Component	Real-World Automotive Equivalent	AUTOSAR Standard / Spec
EcuMonitor	Platform Health Management Watchdog	Adaptive AUTOSAR ara::phm
JsonParser	Parameter Manager / ARXML Configuration	AUTOSAR ECUC
OtaSimulator	Update & Configuration Management (UCM)	Adaptive AUTOSAR UCM